

ETABLISSEMENT

LE GRAND BAZAR

CAHIER DES CHARGES

Application Desktop — Gestion d'une Boutique Généraliste

Filière :

LF – Informatique et Systèmes (IS) – Semestre 6

Auteurs :

SEGNEDEJI K. Emmanuel

KPELOU Presam

Encadrants :

M. ZOMBLEOU Firmin | M. ALLAH – ASSOGBA Firmin

Version : 1.0 **Date :** Mai 2025 **Statut :** Draft

Table des matières

(Clic droit sur cette zone dans Word puis « Mettre à jour les champs » pour afficher le sommaire avec les numéros de page.)

Table des matières	2
1. Présentation du projet	4
1.1 Contexte	4
1.2 Problématique	4
1.3 Objectifs	4
1.4 Périmètre	4
2. Analyse de l'existant	4
2.1 État de l'art	4
2.2 Solutions existantes	5
2.3 Besoins identifiés	5
3. Spécifications fonctionnelles	5
3.1 Besoins fonctionnels	5
Module 1 — Authentification & Utilisateurs	5
Module 2 — Gestion des produits	5
Module 3 — Gestion des stocks	5
Module 4 — Ventes & Facturation	5
Module 5 — Tableau de bord & Rapports	6
3.2 Cas d'usage principaux	6
3.3 Fonctions prioritaires vs optionnelles	6
4. Spécifications techniques	6
4.1 Architecture	6
4.2 Technologies choisies	7
4.3 Contraintes techniques	7
4.4 Performances	7
4.5 Sécurité	7
4.6 Compatibilité	8
5. Planning et organisation	8
5.1 Phases du projet	8
5.2 Répartition des tâches	8
5.3 Livrables attendus	8
6. Gestion des risques	9
6.1 Risques identifiés	9
6.2 Solutions de repli	9
7. Annexes et références	9
7.1 Bibliographie	9

7.2 Glossaire.....	9
7.3 Maquettes (références)	10
7.4 Diagrammes (références) et pistes d'évolution.....	10

1. Présentation du projet

1.1 Contexte

Le projet s'inscrit dans le cadre académique de [votre filière / semestre], au sein de [votre établissement]. Il vise à concevoir et développer une application desktop pour la gestion complète d'une boutique généraliste : produits, stocks, ventes et statistiques.

Les boutiques généralistes font face à des problèmes croissants de gestion manuelle : erreurs de stock, absence de traçabilité, lenteur des opérations de vente et manque d'indicateurs de performance. Une solution informatique légère, fiable et fonctionnant hors-ligne s'impose.

1.2 Problématique

Comment concevoir et développer une application desktop robuste, ergonomique et sécurisée permettant de gérer efficacement les opérations quotidiennes d'une boutique généraliste (produits, stocks, ventes, utilisateurs) sans dépendre d'une connexion internet ni d'un serveur de base de données distant ?

1.3 Objectifs

- Automatiser la gestion des stocks et détecter automatiquement les ruptures
- Faciliter le processus de vente et le calcul des totaux
- Fournir un tableau de bord avec statistiques et graphiques de vente
- Sécuriser l'accès aux données via un système de rôles utilisateurs (Administrateur / Vendeur)
- Permettre l'export et l'impression des données de vente

1.4 Périmètre

L'application couvre les modules suivants :

- Authentification et gestion des utilisateurs
- Gestion des produits et catégories
- Gestion des stocks et alertes de seuil
- Ventes, facturation et impression de reçu
- Tableau de bord, historique et export des ventes

Sont exclus du périmètre actuel : la vente en ligne, la gestion des clients/fournisseurs, la comptabilité avancée, le fonctionnement multi-postes en réseau et l'interconnexion avec des systèmes tiers. Ces points constituent des pistes d'évolution possibles (voir §7.4).

2. Analyse de l'existant

2.1 État de l'art

Plusieurs solutions existent sur le marché pour la gestion de commerce : logiciels ERP (SAP, Sage), solutions cloud (Shopify, Odoo), et applications desktop légères. Ces solutions sont cependant souvent coûteuses, complexes à déployer, ou nécessitent une connexion internet permanente — un frein pour une petite boutique généraliste.

2.2 Solutions existantes

Solution	Limitations
Gestion manuelle (cahiers)	Erreurs, perte de données, lenteur
Excel / Tableurs	Pas de contrôle multi-utilisateur, fragile, pas d'alerte automatique
Solutions ERP payantes	Coût élevé, complexité de déploiement, surdimensionné
Applications cloud	Dépendance à internet, questions de confidentialité des données

2.3 Besoins identifiés

- Interface intuitive adaptée à des opérateurs non techniciens
- Fonctionnement hors-ligne (application desktop, base de données embarquée)
- Gestion multi-utilisateurs avec niveaux d'accès (rôles)
- Système d'alertes automatiques sur les niveaux de stock
- Statistiques de vente et export des données (CSV/Excel)

3. Spécifications fonctionnelles

3.1 Besoins fonctionnels

Module 1 — Authentification & Utilisateurs

- Connexion sécurisée par identifiant / mot de passe
- Mots de passe stockés sous forme hachée (BCrypt), jamais en clair
- Deux rôles : Administrateur et Vendeur
- Création et suppression de comptes utilisateurs (réservé à l'Administrateur)
- Le menu « Utilisateurs » est automatiquement masqué pour le rôle Vendeur

Module 2 — Gestion des produits

- Ajout, modification, suppression de produits
- Classification par catégorie
- Définition du prix d'achat, du prix de vente, de la quantité en stock et du seuil d'alerte
- Recherche instantanée par nom ou catégorie
- Indicateur visuel de statut (« Disponible » / « Stock faible »)

Module 3 — Gestion des stocks

- Suivi de la quantité en stock, produit par produit
- Seuil d'alerte configurable individuellement pour chaque produit
- Écran dédié « Alertes stock » listant les produits sous le seuil
- Mise à jour automatique et transactionnelle du stock lors d'une vente

Module 4 — Ventes & Facturation

- Sélection de produits et constitution d'un panier de vente
- Contrôle du stock disponible avant validation de chaque ligne
- Calcul automatique des sous-totaux et du total de la vente
- Enregistrement transactionnel de la vente (la vente et la mise à jour du stock réussissent ou échouent ensemble)
- Impression d'un reçu de vente détaillé (date, vendeur, produits, total)

Module 5 — Tableau de bord & Rapports

- Chiffre d'affaires du jour et du mois
- Nombre de ventes du jour et nombre de produits en stock faible
- Graphique d'évolution du chiffre d'affaires sur les 7 derniers jours
- Graphique des 5 produits les plus vendus
- Historique complet des ventes, avec détail de chaque vente
- Export CSV (compatible Excel) de l'historique des ventes

3.2 Cas d'usage principaux

Cas d'usage	Acteur principal
S'authentifier	Tous utilisateurs
Enregistrer une vente	Vendeur
Imprimer un reçu de vente	Vendeur
Ajouter / modifier un produit	Administrateur, Vendeur
Consulter les alertes de stock	Tous utilisateurs
Visualiser le tableau de bord	Tous utilisateurs
Exporter l'historique des ventes	Tous utilisateurs
Gérer les utilisateurs	Administrateur

3.3 Fonctions prioritaires vs optionnelles

Priorité	Fonctions
HAUTE (MVP)	Authentification, Produits, Stocks, Ventes
MOYENNE	Tableau de bord, Historique des ventes, Impression du reçu
BASSE (optionnel)	Export CSV/Excel, thème graphique personnalisé, sauvegarde automatique, journal d'audit

4. Spécifications techniques

4.1 Architecture

L'application suit une architecture MVC (Model – View – Controller) découpée en trois couches : Présentation (JavaFX / FXML), Métier (contrôleurs Java) et Persistance (JDBC – SQLite). Chaque écran dispose de son propre fichier FXML et de son contrôleur dédié ; la navigation entre écrans est centralisée dans un contrôleur de mise en page principal (MainLayoutController).

4.2 Technologies choisies

Couche	Technologie	Détail / Justification
Langage	Java (JDK 17+)	Langage orienté objet robuste, portable et bien documenté
Interface utilisateur	JavaFX 21 + FXML	Framework graphique moderne pour desktop ; séparation vue/contrôleur
Icônes	Ikonli (pack Feather)	Icônes vectorielles cohérentes, sans dépendre d'images bitmap
Base de données	SQLite	SGBD embarqué, sans serveur, adapté à une application mono-poste
Connecteur BD	JDBC (sqlite-jdbc)	API standard Java pour la connexion à la base de données
Sécurité mots de passe	jBCrypt	Hachage sécurisé et salé des mots de passe utilisateurs
Build / dépendances	Maven	Gestion des dépendances et du cycle de vie du projet
Impression	API javafx.print	Génération et impression directe du reçu de vente

4.3 Contraintes techniques

- Compatibilité multiplateforme : Windows, macOS, Linux (JavaFX)
- Java JDK 17 minimum requis sur le poste
- Base de données SQLite embarquée (fichier local dans le dossier database/), aucun serveur requis
- Application mono-poste dans la version actuelle (pas d'accès réseau concurrent à la base)
- Interface en français

4.4 Performances

- Temps de réponse des opérations courantes : inférieur à 2 secondes
- Démarrage de l'application : inférieur à 5 secondes
- Gestion fluide d'un catalogue de plusieurs milliers de références produits

4.5 Sécurité

- Hachage des mots de passe avec BCrypt (jamais de mot de passe en clair en base)
- Contrôle d'accès basé sur les rôles (RBAC) : Administrateur / Vendeur
- Requêtes SQL préparées (PreparedStatement) pour prévenir les injections SQL

- Transactions SQL pour garantir la cohérence entre une vente et la mise à jour du stock

4.6 Compatibilité

- JavaFX 21 ou supérieur
- SQLite (pilote sqlite-jdbc 3.45+)
- IDE recommandé : NetBeans ou IntelliJ IDEA avec support Maven
- Résolution d'écran minimale recommandée : 1366 × 768 pixels

5. Planning et organisation

5.1 Phases du projet

Phase	Description
Phase 1 — Analyse	Recueil des besoins, étude de l'existant, rédaction du cahier des charges
Phase 2 — Conception	Modélisation (cas d'usage, classes), conception de la base de données, maquettes d'interface
Phase 3 — Développement	Implémentation des modules (produits, stocks, ventes, tableau de bord, utilisateurs)
Phase 4 — Tests & intégration	Tests fonctionnels, correction des anomalies, packaging de l'application
Phase 5 — Livraison	Documentation, démonstration, soutenance

5.2 Répartition des tâches

[À compléter selon la composition de votre équipe.]

Tâche	Responsable
Analyse des besoins & cahier des charges	SEGNEDEJI Komivi Emmanuel
Conception de la base de données	KPELOU Présam
Développement Back-end (Java, DAO)	SEGNEDEJI Komivi Emmanuel
Développement Front-end (JavaFX / FXML)	KPELOU Présam
Tests & documentation	SEGNEDEJI & KPELOU

5.3 Livrables attendus

- Cahier des charges (présent document)
- Dossier de conception (diagrammes, modèle de base de données)
- Code source commenté (dépôt Git)
- Manuel utilisateur
- Support de présentation de soutenance

6. Gestion des risques

6.1 Risques identifiés

Risque	Probabilité	Impact	Plan de mitigation
Manque de maîtrise de JavaFX/FXML	Moyenne	Haut	Tutoriels officiels, exercices pratiques préalables
Erreurs de chemins de ressources (FXML/CSS/images) au chargement	Moyenne	Moyen	Centraliser le chargement des ressources dans une méthode unique et testée (ex. <code>resolveResource</code>), tester chaque écran après ajout
Incohérence des données lors d'une vente (stock non mis à jour)	Faible	Haut	Utilisation de transactions SQL (<code>commit/rollback</code>) pour lier vente et mise à jour du stock
Retard dans le planning	Moyenne	Moyen	Suivi régulier de l'avancement, priorisation des fonctions du MVP
Indisponibilité d'un membre de l'équipe	Faible	Moyen	Partage du code via Git, documentation continue du projet

6.2 Solutions de repli

- Simplification de certaines fonctionnalités si le planning est dépassé (priorisation MVP)
- Conservation de SQLite comme choix définitif si une migration vers un SGBD serveur s'avère trop coûteuse en temps
- Découpage en livraisons intermédiaires pour permettre une évaluation progressive

7. Annexes et références

7.1 Bibliographie

- Documentation officielle Java : <https://docs.oracle.com/en/java/>
- Documentation JavaFX : <https://openjfx.io/>
- Documentation SQLite : <https://www.sqlite.org/docs.html>
- Pilote sqlite-jdbc : <https://github.com/xerial/sqlite-jdbc>
- Bibliothèque d'icônes Ikonli : <https://kordamp.org/ikonli/>
- jBCrypt : <https://www.mindrot.org/projects/jBCrypt/>

7.2 Glossaire

Terme	Définition
JDBC	Java Database Connectivity — API Java de connexion aux SGBD
JavaFX	Framework Java pour le développement d'interfaces graphiques desktop
FXML	Format XML utilisé par JavaFX pour décrire une interface graphique

Terme	Définition
SQLite	Système de gestion de base de données relationnelle embarqué, sans serveur
BCrypt	Algorithme de hachage sécurisé utilisé pour stocker les mots de passe
MVC	Model-View-Controller — patron de conception architectural
MVP (fonctionnel)	Minimum Viable Product — version minimale livrable
RBAC	Role-Based Access Control — contrôle d'accès basé sur les rôles
CDC	Cahier Des Charges

7.3 Maquettes (références)

Les écrans principaux de l'application sont les suivants :

- Écran de connexion
- Tableau de bord principal (statistiques et graphiques)
- Interface de gestion des produits
- Interface de vente (nouvelle vente)
- Interface des alertes de stock
- Historique des ventes
- Gestion des utilisateurs

7.4 Diagrammes (références) et pistes d'évolution

Diagrammes à produire dans le dossier de conception :

- Diagramme de cas d'usage global
- Diagramme de classes (modèle métier : Produit, Utilisateur, Vente, LigneVente)
- Diagramme de séquence (processus de vente)
- Modèle Entité-Association (base de données)

Pistes d'évolution possibles hors périmètre actuel :

- Gestion des clients et fournisseurs
- Export PDF des rapports
- Mode réseau multi-postes avec base de données serveur
- Journal d'audit des actions sensibles